Elegant Solution

คู่มือการใช้งาน การติดตั้ง และการบริหารจัดการระบบ

Complete System, Installation & Technical Manual

Version 0.5.0 | 12 พฤศจิกายน 2026

อ้างอิง repository ที่ commit ada86dadbaa1e4711cca99c56c1f1d6d83b2174a

วิธีอ่านคู่มือนี้

คู่มือนี้ใช้โค้ดใน branch feature/cross-platform-tracking-agent-exam ณ commit ที่ระบุบนปกเป็นหลักฐานหลัก เอกสารประวัติและผลทดสอบที่ผู้ดูแลรายงานแยกจากพฤติกรรมที่ยืนยันในโค้ด ทุกตัวอย่างใช้ข้อมูลสมมติ คำสั่งที่แตะ production ต้องผ่านขั้นอนุมัติและสำรองข้อมูลตามบทที่ 9

ป้ายสถานะ: **IMPLEMENTED** = มีโค้ด; **AUTOMATED TESTED** = มีผลตรวจอัตโนมัติที่รายงาน; **VERIFIED ON REAL HARDWARE** = มีผลจากเครื่องจริงที่ผู้ใช้รายงาน; **NOT YET VERIFIED** = ยังไม่มีหลักฐาน; **PARTIAL IMPLEMENTATION** = มีโค้ดบางส่วน; **NOT IMPLEMENTED** = ไม่พบการทำงานนั้นในโค้ดที่ตรวจ

หมายเหตุ ผลทดสอบ macOS ล่าสุดมาจากคำบอกและผล Terminal ที่ผู้ใช้ส่งในบทสนทนา: ติดตั้ง/activate, reboot แล้ว UI เปิดเอง, เครื่องเดิม Online ไม่มีรายการซ้ำ, จากนั้นถอนด้วย maintenance code และตรวจว่า receipt/service/UI หายไป การ login นักศึกษาและ foreground activity ยังไม่มีผลทดสอบบนเครื่องจริงในหลักฐานชุดนี้

สารบัญ

วิธีอ่านคู่มือนี้ i

I คู่มือผู้ใช้ 1

1 รู้จัก Elegant Solution 2

2 บทบาทและสิทธิ์ 3

3 สมัครบัญชีและเข้าสู่ระบบ 4

4 คู่มือนักศึกษา 5

5 คู่มือ Guest 7

6 คู่มือ Admin 8

7 คู่มือ Super Admin 9

II การติดตั้งและบริหารระบบ 10

8 สถาปัตยกรรมติดตั้งใช้งาน 11

9 Backend: ติดตั้งและ deploy 12

10 Tracking Agent 0.5.0 15

11 Batch Enrollment และรหัสเปิดใช้งาน 16

12 ติดตั้งและถอน Agent บน Windows 17

13 ติดตั้งและถอน Agent บน macOS 18

14 ติดตั้งและถอน Agent บน Ubuntu 19

15 บำรุงรักษาและแก้ปัญหา 20

III อ้างอิงเทคนิคสำหรับผู้พัฒนา 21

16 สถาปัตยกรรมและเทคโนโลยี 22

17 ฐานข้อมูลและความสัมพันธ์ 23

18 กระบวนการ OCR ตารางเรียน 24

19 Face AI และการยืนยันใบหน้า 25

20 QR, access token และ Gate 26

21 รหัสและ token สำคัญ 27

22 Room Availability และ Entitlement 28

23 Agent API และ Monitoring 29

24 ความปลอดภัย ความเป็นส่วนตัว และข้อจำกัด 31

A เช็กลิสต์ปฏิบัติงาน 32

B คำสั่งอ้างอิงแบบอ่านอย่างเดียว 34

C อภิธานศัพท์และสถานะ 35

D บัญชีประเด็นเอกสารและแหล่งอ้างอิง 36

ภาค I

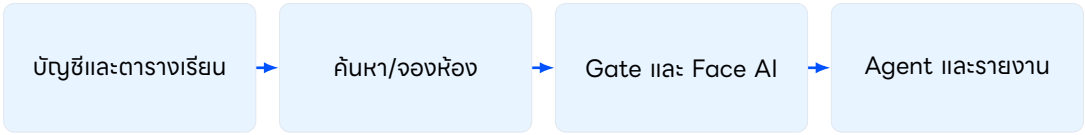
คู่มือผู้ใช้

บทที่ 1

รู้จัก Elegant Solution

Elegant Solution เป็นระบบจัดการการใช้ห้องปฏิบัติการที่เชื่อมบัญชีผู้ใช้ ตารางเรียน การค้นหาห้อง การจอง สิทธิ์เข้าห้อง จุดตรวจหน้าห้อง และสถานะเครื่องคอมพิวเตอร์ ผู้ใช้ทำงานผ่านเว็บ; ผู้ดูแลกำหนดห้อง อนุมัติคำขอ ตรวจประวัติ และติดตั้ง Tracking Agent บนเครื่องแล็บ จุดมุ่งหมายของโครงการคือให้ข้อมูลการใช้ห้องและเครื่องสัมพันธ์กันในระบบเดียว โดยไม่อ้างผลสำรวจหรือการทดลองที่ไม่มีหลักฐาน

ขอบเขตสำคัญมีสองชั้น: เว็บและ Face AI ทำงานบนเซิร์ฟเวอร์; Tracking Agent ทำงานบนเครื่องแล็บแต่ละเครื่อง การมี Agent Online หมายถึงเครื่องส่ง heartbeat ได้ ไม่ได้แปลว่ามีนักศึกษาผ่านการตรวจสิทธิ์แล้ว และซอฟต์แวร์ยืนยันการเข้าห้องไม่ได้พิสูจน์ว่ากลอนไฟฟ้าปลดล็อกจริง



รูปที่ 1.1: ภาพรวมระบบ (แผนภาพอธิบายจากโค้ด)

ความสามารถหลักและขอบเขต

ระบบมีเว็บสำหรับ STUDENT, GUEST, ADMIN, SUPER_ADMIN; ตารางเรียน, OCR, Room Finder, booking, token QR, face enrollment/verification, Admin Tracking และ batch enrollment. หน้า Contact/Survey เป็นการแสดงผลบางส่วนและไม่ควรอ้างว่ารับข้อมูลจริงโดยไม่ตรวจโค้ดปลายทาง เว็บไซต์ blacklist ใน Agent รุ่น 0.5.0 ไม่ได้ยืนยันการบังคับใช้ครบทุก OS. การบังคับ login ก่อนใช้งาน desktop macOS ยังไม่ใช่พฤติกรรมของ Agent ปัจจุบัน

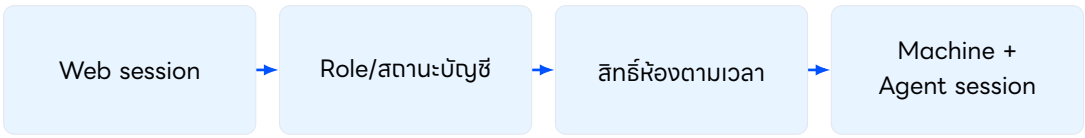
หลักฐานใน repository: web/src/lib/permissions.ts;web/src/lib/room-entitlement.ts;agent/src/session_ui.py

บทที่ 2

บทบาทและสิทธิ์

สิทธิ์ไม่ได้มาจากชื่อหน้าที่ UI แสดงอย่างเดียว Backend ตรวจ session, role, account status และในกรณีห้อง ตรวจ entitlement จากข้อมูลจริง การ login เว็บ, การยืนยันตัวเครื่องด้วย API key และ Agent user session เป็นคนละสิ่ง

รายการ	SUPER_ADMIN	ADMIN	STUDENT	GUEST
เว็บและโปรไฟล์	ได้	ได้	ได้	ได้เมื่อ login
เมนู Admin ยกเว้น System Control	ได้	ได้	ไม่ได้	ไม่ได้
System Control และ Deployment Batch	ได้	ไม่ได้	ไม่ได้	ไม่ได้
Room Finder	ดูได้	ดูได้	ดูได้	เฉพาะ approved และห้องที่เปิดให้ Guest
จองห้องด้วยตนเอง	ไม่ได้	ไม่ได้	ได้	เฉพาะ approved
ลงทะเบียนใบหน้า	ได้	ได้	ได้	ไม่ได้
Agent user login	มีบัญชี approved และสิทธิ์ห้อง ณ เวลาใช้งาน	เช่นเดียวกัน	เช่นเดียวกัน	เช่นเดียวกัน แต่ guest ไม่มี face enrollment



รูปที่ 2.1: ชั้นการตรวจสิทธิ์ (แผนภาพอธิบาย)

การสมัครผ่านหน้าเว็บสร้าง GUEST/PENDING; เจ้าหน้าที่อนุมัติเป็น APPROVED หรือปฏิเสธเป็น REJECTED. นักศึกษา Google ใหม่ใช้โดเมน bumail.net ที่ยืนยันอีเมลแล้ว และถูกสร้างเป็น STUDENT/APPROVED. บัญชี ADMIN/SUPER_ADMIN ที่มีอยู่ใช้ Google ได้เมื่ออีเมลยืนยัน โดย role ยึดฐานข้อมูล. ดูสิทธิ์หน้า Admin จาก server action/API ไม่ใช่รายการเมนูอย่างเดียว

หลักฐานใน repository: web/src/lib/permissions.ts;web/src/lib/auth.ts;web/src/lib/google-sign-in.ts;web/src/lib/admin-nav.ts; web/src/app/api/agent/auth/route.ts

บทที่ 3

สมัครบัญชีและเข้าสู่ระบบ

สมัคร Guest

1. เปิด /register กรอกชื่อ อีเมล รหัสผ่าน และเบอร์โทรถ้ามี; หน้าฟอร์มตรวจรหัสผ่านอย่างน้อย 6 ตัวอักษรและยืนยันซ้ำ
2. ส่งแบบฟอร์ม ระบบตรวจอีเมลซ้ำและเก็บรหัสผ่านแบบ bcrypt; บัญชีใหม่มี role GUEST สถานะ PENDING
3. รอผู้ดูแลตรวจที่ /admin/users. หากยังไม่อนุมัติ Guest ใช้ Room Finder/booking ไม่ได้; หากถูกปฏิเสธให้ติดต่อผู้ดูแล

เข้าสู่ระบบและจัดการบัญชี

ที่ /login ใช้อีเมล/รหัสผ่าน หรือ Google ตามกฎโดเมนข้างต้น หากข้อมูลผิดจะไม่สร้าง web session. หน้า /profile แก้ชื่อ/โทรศัพท์และเปลี่ยนรหัสผ่านด้วยรหัสเดิมได้ นักศึกษาผูก student ID 10 หลักครั้งแรก; หากต้องแก้ภายหลังให้ติดต่อ admin. ไม่พบ workflow ส่งอีเมล reset password อัตโนมัติในโค้ดที่ตรวจ การ logout เว็บไม่ใช่การ logout Agent session บนเครื่องแล็บ

ข้อควรระวัง บัญชีที่ login ด้วย Google และมีรหัสผ่านสุ่มภายใน ไม่ควรอ้างว่าผู้ใช้รู้รหัสผ่านนั้น การเปลี่ยนรหัสผ่านที่หน้าโปรไฟล์ต้องผ่านรหัสปัจจุบันตามโค้ด

หลักฐานใน repository: web/src/actions/auth.ts;web/src/lib/auth.ts;web/src/actions/profile.ts;web/src/app/(auth)/register/page.tsx

บทที่ 4

คู่มือนักศึกษา

Dashboard และตารางเรียน

หลัง login เข้า /dashboard เพื่อดูข้อมูลห้อง ตารางเรียน และรายการที่เกี่ยวข้องกับบัญชี ตารางเรียนสัมพันธ์กับภาคการศึกษาที่ระบบกำหนด; ถ้าไม่พบรายการให้ตรวจ semester และ student ID ก่อน ระบบใช้วัน/เวลาแบบ Asia/Bangkok ในการให้สิทธิ์ห้อง ผู้ใช้ควรตรวจความถูกต้องของวิชา ห้อง วัน และช่วงเวลาเสมอ

นำเข้าตาราง BU Mobile ด้วย OCR

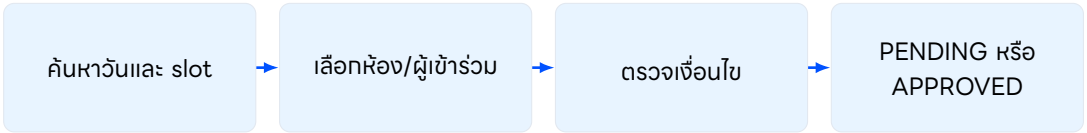
1. เตรียมภาพหน้าจอตารางเรียนที่ตัวอักษรชัด ครบชื่อวิชา วัน เวลา และห้อง; เปิดแผง OCR ใน Dashboard
2. เลือกภาพ ระบบโหลด tesseract.js@5.1.1 ใน browser แล้วอ่านข้อความจากภาพ
3. ตรวจ preview ที่แยกรายการพร้อมคำเตือน/ข้อผิดพลาด; ตรวจ course code, section, ชื่อวิชา, วัน, slot และห้อง
4. แก้ข้อมูลที่จำเป็นใน preview ตาม UI แล้วสั่ง import เฉพาะรายการที่พร้อม ระบบรวมรายการซ้ำที่ parser พบ; ตรวจตารางที่บันทึกแล้วอีกครั้ง

หมายเหตุ OCR เป็นการช่วยอ่านภาพ ไม่ใช่หลักฐานว่าข้อมูลถูกต้องทุกช่อง ข้อความดิบและ parser debug มีไว้ตรวจปัญหาใน UI; อย่าใช้ภาพตารางจริงของผู้อื่นในคู่มือหรือการสาธิต

ค้นหาห้องและจอง

เปิด Room Finder จาก Dashboard เลือกวันที่, slot, จำนวนผู้ใช้, ความสามารถที่ต้องการ, อาคารและชั้นตามตัวกรองที่ UI มี ผล Available Now ขึ้นกับช่วงเวลาปัจจุบัน ตารางเรียน การจอง หน้าต่างเปิดให้จอง การปิดห้อง และความจุ ห้องที่เปิด booking แต่ยังไม่มี availability window จะถูกปฏิเสธแบบ fail-closed

เลือกห้องและ slot → ตรวจผู้เข้าร่วม → ยืนยันคำขอ. การจองเดี่ยวคือผู้เข้าร่วมหนึ่งคน; booking แบบกลุ่มใช้ snapshot รายชื่อสมาชิก ณ เวลาสร้าง booking. สถานะ PENDING ต้องรออนุมัติ; APPROVED ให้สิทธิ์ตามช่วงเวลาที่อนุมัติ การสร้าง booking ยังไม่สร้าง session, access token หรือสั่งประตูเปิด หน้าสร้าง Group QR ใน Dashboard ถูกปิดด้วย ENABLE_GROUP_UI = false แม้มีโค้ดกลุ่มอยู่



รูปที่ 4.1: การจองห้องของนักศึกษา (แผนภาพอธิบาย)

เข้าห้องและประวัติ

สิทธิ์ปัจจุบันมาจาก CLASS, booking ที่อนุมัติและมีชื่อผู้ใช้ใน participant snapshot, หรือ LEGACY_TOKEN ที่ออกให้ตนเอง. CLASS เริ่มก่อนเวลาเรียน 30 นาที. ที่ Gate สามารถสแกน QR token หรือใช้ face verification ตามช่องทางที่เปิดใช้; successful face check-in/check-out บันทึก Session. QR token เดิมมี validation ของตนเองตามบท 20; อย่าสมมติว่า QR ทุกอันให้สิทธิ์ผ่าน entitlement engine แบบเดียวกัน. ตรวจประวัติใน Dashboard/รายงานตามสิทธิ์

หลักฐานใน repository: web/src/components/ocr-timetable-panel.tsx;web/src/lib/availability.ts;web/src/lib/room-booking.ts;web/src/lib/room-entitlement.ts;web/src/app/(app)/dashboard/dashboard-client.tsx

บทที่ 5

คู่มือ Guest

Guest สมัครที่ /register แล้วรออนุมัติจาก Admin. หลังสถานะ APPROVED จึงค้นหาและจองห้องที่ตั้ง guestAccess=true และไม่ใช่ staffOnly. การเลือกเวลาและอนุมัติ booking ใช้หลักเดียวกับนักศึกษา แต่ Guest ไม่มีเส้นทางลงทะเบียนใบหน้าใน /api/face/enroll; การสาธิต Gate ด้วยใบหน้าของ Guest จึงไม่ใช่ workflow ที่รองรับเต็มรูปแบบในรุ่นนี้

1. ตรวจสถานะบัญชีและ login เว็บ
2. เปิด Room Finder; ถ้าห้องไม่ปรากฏ ให้ Admin ตรวจ guestAccess, staffOnly, เวลาเปิด และความจุ
3. ส่ง booking และรออนุมัติหากนโยบายห้องเป็น MANUAL
4. ดูสถานะคำขอและติดต่อ Admin ถ้าถูกปฏิเสธ; อย่าใช้ QR หรือบัญชีของผู้อื่นแทน

หลักฐานใน repository: web/src/lib/permissions.ts;web/src/actions/auth.ts;web/src/app/api/face/enroll/route.ts;web/src/lib/room-booking.ts

บทที่ 6

คู่มือ Admin

เมนูของ Admin ประกอบด้วย ภาพรวม, จัดการผู้ใช้, จัดการห้อง, คำขอจองห้อง, ตารางจองห้อง, นำเข้าตารางเรียน, ติดตามการใช้งาน, System Health, ประวัติการใช้งาน, รายงาน และวิเคราะห์ข้อมูล. หน้า /admin/system สงวนให้ SUPER_ADMIN ตามการนำทางและ server action

งานประจำวัน

1. เปิด /admin/dashboard ตรวจภาพรวม แล้วไป /admin/users เพื่ออนุมัติ Guest/จัดการบัญชีตามสิทธิ์
2. ที่ /admin/rooms ตรวจรหัสห้อง สถานะ ความจุ capabilities, guest access, staff-only, นโยบาย AUTO/MANUAL, availability windows และ closures
3. ที่ /admin/bookings ตรวจคำขอ PENDING, ผู้ร่วมจอง, วัน/slot และความขัดแย้งก่อนอนุมัติหรือปฏิเสธ; ดู /admin/calendar ประกอบ
4. ที่ /admin/schedule นำเข้าตารางเรียนตามภาคเรียน และตรวจแถวผิดพลาดก่อนใช้งาน Gate
5. ที่ /admin/tracking ดูเครื่อง Online, ผู้ใช้ปัจจุบัน และประวัติกิจกรรม; Online ของเครื่องไม่ยืนยันว่ามีนักศึกษา login Agent
6. ตรวจ /admin/diagnostics, /admin/logs, /admin/reports, /admin/analytics เพื่อวิเคราะห์เหตุการณ์และส่งออกข้อมูลตามที่หน้าเว็บรองรับ

Admin Tracking มีเมนูสร้าง maintenance code ผูกเครื่องและ action; code มีอายุ 5 นาทีและใช้ครั้งเดียว การแก้ website blacklist ผ่านหน้า Tracking เป็นการตั้งค่า backend; การบังคับใช้จริงขึ้นกับ Agent/OS และยังไม่ควรอ้างว่าครบทุกแพลตฟอร์ม

หลักฐานใน repository: web/src/lib/admin-nav.ts;web/src/actions/agent.ts;web/src/app/admin/tracking/tracking-client.tsx;web/src/lib/room-admin.ts

บทที่ 7

คู่มือ Super Admin

SUPER_ADMIN มีเมนู Admin ทั้งหมดและหน้า /admin/system เพิ่มเติม ใช้ตั้งค่าระบบและจัดการ Tracking Agent Deployment. การสร้าง batch ต้องเลือกห้องที่ active และจำนวนเต็ม 1–100 เครื่องต่อ batch. CSV ที่ดาวน์โหลดได้ครั้งเดียวควรเก็บในพื้นที่จำกัดสิทธิ์และแจกหนึ่ง code ต่อหนึ่งเครื่อง การตั้งค่า SUPER_ADMIN คนแรกเป็นขั้นตอนที่ต้องจัดการโดยผู้ดูแลระบบที่ได้รับมอบหมาย: seed script ที่ตรวจสร้างบัญชีตัวอย่าง role ADMIN ไม่ได้สร้าง SUPER_ADMIN. การสร้าง/เลื่อนสิทธิ์ผู้ดูแลผ่านเว็บต้องมี SUPER_ADMIN อยู่ก่อน จึงไม่ควรอ้างว่ามี bootstrap อัตโนมัติหรือรัน seed ตัวอย่างบน production

1. เปิด /admin/system ส่วน Tracking Agent Deployment; เลือกห้องและจำนวนเครื่อง
2. สร้าง batch; ตรวจ room code, จำนวนที่ร้องขอ และเวลา expiry
3. ดาวน์โหลด CSV ใน browser ทันที เก็บอย่างปลอดภัย; แจก code ตาม machine code ที่จองไว้
4. ติดตั้ง PKG/EXE/DEB ให้ตรง OS; เปิด UI และกรอก code
5. กลับมาดูสถานะ PENDING, ACTIVATED, REVOKED, EXPIRED; ตรวจ Admin Tracking เพิ่มเติม

Batch code มีอายุ 7 วัน; code เดี่ยวจาก Admin Tracking มีอายุ 10 นาที. การ revoke batch ปิดเฉพาะ code ที่ยังไม่ใช้ ไม่ได้เพิกถอน machine ที่ activate ไปแล้ว. Installer download ต้องตั้ง AGENT_INSTALLER_DIR และ stage artifact บน backend ก่อน หน้า System Control ยังมีตัวเลือกการตั้งค่าระบบอื่นตามโค้ด; เปลี่ยนค่าเฉพาะเมื่อทราบผลกระทบและบันทึกเหตุผล

หลักฐานใน repository: web/src/actions/agent-deployments.ts;web/src/lib/agent-deployment-batches.ts;web/src/lib/agent-activation.ts;web/src/app/api/admin/agent/installers/[platform]/route.ts;web/prisma/seed.ts

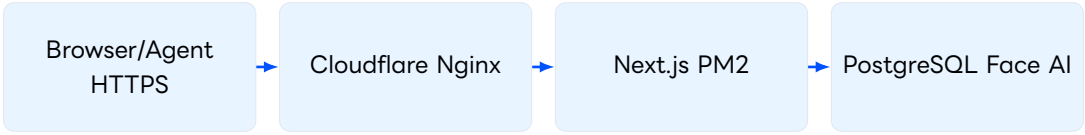
ภาค II

การติดตั้งและบริหารระบบ

บทที่ 8

สถาปัตยกรรมติดตั้งใช้งาน

Production ที่เคยตรวจบนเซิร์ฟเวอร์ Ubuntu 20.04.6 ใช้ Cloudflare เป็น HTTPS สาธารณะ, Nginx proxy ไป Next.js พอร์ต 3000, PM2 ชื่อ elegant, PostgreSQL 16 ใน Docker container elegant-postgres และ Face AI แยกพอร์ต 8000. นี่เป็น topology ที่บันทึกไว้ใน runbook; คู่มือนี้ไม่ได้ SSH เข้าเซิร์ฟเวอร์เพื่อยืนยันสถานะสด เครื่อง-



รูปที่ 8.1: เส้นทาง production ที่บันทึกไว้ (แผนภาพสถาปัตยกรรม)

แล็บ Windows/macOS/Ubuntu รัน Tracking Agent และติดต่อ /api/agent/* ทาง HTTPS. Backend เก็บแพ็กเกจ installer ใน directory ภายนอก web root เพื่อให้ Admin ดาวน์โหลด; **ไม่ต้องติดตั้ง Tracking Agent บนเซิร์ฟเวอร์ backend**. Machine service กับ UI อยู่บนเครื่องปลายทาง

เครือข่ายและขอบเขต

ตรวจ DNS/TLS ไป https://elegant.fafromh0me.com; Nginx ส่งต่อ localhost:3000; PostgreSQL ผูก localhost. Runbook ระบุ Face AI พอร์ต 8000 บน server และการตรวจเดิมพบ origin พอร์ต 3000/8000 เข้าถึงจากภายนอกได้ จึงต้องตรวจ firewall/reverse proxy และวางแผนจำกัดการเข้าถึงใน change แยก Agent ใช้เฉพาะ HTTPS backend ไม่ต้องติดต่อ PostgreSQL หรือ Face AI โดยตรง

หลักฐานใน repository: docs/PRODUCTION_PM2_DEPLOYMENT.md;agent/README.md;web/src/app/api/agent/heartbeat/route.ts

บทที่ 9

Backend: ติดตั้งและ deploy

ก่อน deploy

สภาพ production ที่บันทึกล่าสุดมี Node.js v24.13.1, npm 11.8.0, PM2 5.3.0, global pnpm 8.8.0; release ใหม่ใช้ npx --yes pnpm@9.15.9 แบบแยก. RAM ว่างประมาณ 2.4 GiB, swap ถูกใช้ 1.8/4 GiB และ disk ว่าง 11 GB ณ preflight; ค่าจริงต้องตรวจซ้ำ. Repository production มีไฟล์แก้และ untracked จึงใช้ Git worktree/release directory ใหม่ ไม่ overwrite checkout เดิม

```
# ตรวจอ่านอย่างเดียวบน server ก่อนเริ่มขั้นเปลี่ยนระบบ
node --version
pm2 list
free -h
df -h /home/lamy
sudo docker exec elegant-postgres sh -lc \
  'pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB"'
git -C /home/lamy/Elegant-Solutions status --short
```

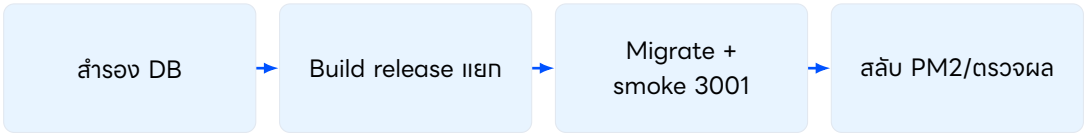
ตั้ง release จาก full commit SHA ที่อนุมัติแล้วตาม docs/PRODUCTION_PM2_DEPLOYMENT.md. คัดลอก runtime .env แบบรักษาสิทธิ์ 600 และตรวจด้วย cmp -s; ห้ามพิมพ์ค่า secret. ตรวจ DATABASE_URL, AUTH_SECRET, AUTH_URL และ environment ที่เกี่ยวข้องว่ามีค่าและไม่ขัดกับ PM2 เดิมโดยไม่แสดงค่า. AGENT_INSTALLER_DIR ชี้ directory ของ artifacts ซึ่งไม่อยู่ใน web root

ข้อควรระวัง ห้ามรัน web/prisma/seed.ts กับ production เพื่อสร้าง Super Admin: script นี้สร้างบัญชีตัวอย่างด้วย credential ที่ฝังไว้และไม่ได้สร้าง SUPER_ADMIN. การเตรียมบัญชีผู้ดูแลคนแรกต้องทำตามกระบวนการอนุมัติและข้อมูลจริงของหน่วยงาน

ลำดับหลังอนุมัติ production change

1. สำรอง PostgreSQL ด้วย pg_dump -Fc; ตรวจ checksum และ pg_restore --list; ถ้าเป็นไปได้ restore ทดสอบสำเนาแยก
2. สร้าง isolated release จาก commit ที่ตรวจ migration history แล้ว; ติดตั้ง dependencies ด้วย pnpm 9.15.9; รัน Prisma generate และ build โดยจำกัด worker/heap ตาม runbook
3. ตรวจ prisma migrate status. หากแสดง pending เฉพาะรายการที่คาดจึงรัน prisma migrate deploy; ห้ามใช้ reset/db push แบบทำลายข้อมูล
4. เริ่ม build ใหม่แบบ standalone บนพอร์ต 3001 เพื่อตรวจ /api/liveness; หลังผ่านจึงสลับ PM2 ตาม runbook และตรวจ HTTPS public endpoint
5. บันทึก SHA, PM2 cwd/command, สถานะ migrations และ health checks; เก็บ release เดิมเพื่อ rollback

ข้อควรระวัง บล็อกต่อไปเป็นตัวอย่างคำสั่งสำหรับ release ที่แยกแล้ว ไม่ใช่คำสั่งให้รันทันทีบน production. ค่าภายใน .env และ backup ห้ามใส่ใน Git/คู่มือ



รูปที่ 9.1: การปล่อย backend (แผนภาพอธิบาย)

```
cd "$RELEASE/web"
NODE_ENV=development npx --yes pnpm@9.15.9 install --frozen-lockfile
npx --yes pnpm@9.15.9 exec prisma generate --config ./prisma.config.ts
CIRCLE_NODE_TOTAL=2 NODE_OPTIONS=--max-old-space-size=1536 \
  npx --yes pnpm@9.15.9 build
# หลังอนุมัติและมี backup ที่ตรวจแล้วเท่านั้น:
npx --yes pnpm@9.15.9 exec prisma migrate status --config ./prisma.config.ts
npx --yes pnpm@9.15.9 exec prisma migrate deploy --config ./prisma.config.ts
```

คำสั่งสำรองและตรวจ archive บน server ใช้ sudo docker exec ตามสิทธิ์ที่ตรวจแล้ว. การอ่านรายการใน archive ไม่เท่ากับการ restore ทดสอบ; ก่อน cutover ต้องมีผล restore แยกในสภาพแวดล้อมที่ควบคุมได้

```
# PRODUCTION CHANGE: ทำหลังอนุมัติเท่านั้น
BACKUP_DIR=/home/lamy/elegant-db-backups
install -d -m 700 "$BACKUP_DIR"
BACKUP="$BACKUP_DIR/elegant_$(date +%Y%m%d_%H%M%S).dump"
umask 077
sudo docker exec elegant-postgres sh -lc \
  'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" -Fc' > "$BACKUP"
pg_restore --list "$BACKUP" >/dev/null
sha256sum "$BACKUP"
```

การ start build ใหม่ใช้ Node รัน standalone server.js ไม่พึ่ง global pnpm 8. Smoke test ที่พอร์ต 3001 ก่อน แล้วจึง cutover พอร์ต 3000 ผ่าน PM2. ตรวจจาก localhost และ public HTTPS; คำสั่งจริงต้องใช้ STANDALONE_SERVER และ STANDALONE_DIR ที่ตรวจแล้วจาก release ตาม runbook

```
# PRODUCTION CHANGE: เฉพาะหลัง migrate/build/smoke ผ่าน
pm2 stop elegant
PORT=3000 HOSTNAME=0.0.0.0 pm2 start "$STANDALONE_SERVER" \
  --name elegant-v050 --interpreter "$(command -v node)" \
  --cwd "$STANDALONE_DIR"
curl --fail --silent --show-error \
  http://127.0.0.1:3000/api/liveness
curl --fail --silent --show-error \
  https://elegant.fafromh0me.com/api/liveness
pm2 list
pm2 save
```

Rollback แอปคือกลับไป PM2 release เดิมตาม runbook; schema ใหม่ถูกออกแบบให้ backward compatible กับแอปเดิม แต่การกู้ DB คือขั้นแยกที่ต้องวางแผน downtime/ข้อมูลใหม่หลัง backup. อย่าทับฐานข้อมูลสดด้วย dump อัตโนมัติ อ่านคำสั่ง PM2 cutover และ rollback แบบเต็มจาก runbook ก่อนปฏิบัติ

```
# APP ROLLBACK: หลังอนุมัติแก้เหตุขัดข้อง
pm2 stop elegant-v050 >/dev/null 2>&1 || true
pm2 restart elegant
curl --fail --silent --show-error \
  https://elegant.fafromh0me.com/
pm2 save
```

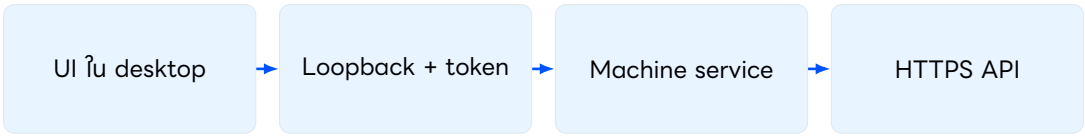
หากต้องกู้ฐานข้อมูล ให้ restore backup ลงฐาน ใหม่ ก่อน เปรียบเทียบ migration/จำนวน records และขออนุมัติแยกก่อนเปลี่ยน DATABASE_URL. การ restore ทับ production โดยตรงหลังมี writes ใหม่ทำข้อมูลหลังเวลาสำรองสูญหาย

หลักฐานใน repository: docs/PRODUCTION_PM2_DEPLOYMENT.md;web/next.config.ts;web/prisma/schema.prisma

บทที่ 10

Tracking Agent 0.5.0

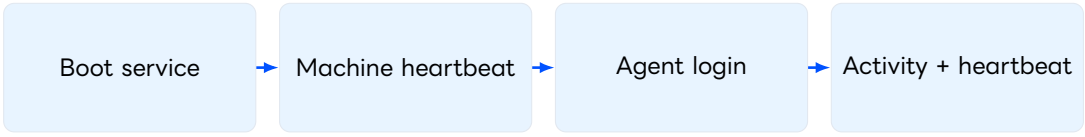
Agent แยก machine_service.py จาก session_ui.py. Service เก็บ machine identity/API key ในพื้นที่ระบบ ส่ง heartbeat ทุก 10 วินาทีแม้ไม่มีนักศึกษา login และ sync config. UI ใน graphical session ใช้ code เปิดใช้งาน, login ผู้ใช้ และอ่าน foreground app/title เฉพาะเมื่อ Agent user session active. UI ติดต่อ service ผ่าน loopback 127.0.0.1:18767 โดย local bridge token; service ส่งข้อมูลไป backend



รูปที่ 10.1: องค์ประกอบ Agent (แผนภาพอธิบาย)

วงจร session และความเป็นส่วนตัว

Activation ใช้ code หกหลักครั้งเดียวและได้ machine code/API key ซึ่งบันทึกแบบสิทธิ์จำกัด. Agent login ตรวจ email/password บัญชี APPROVED และ entitlement ของห้องที่ผูกเครื่อง; backend ออก session token ที่ผูก user, machine, room และเวลาสิ้นสุด. เมื่อ session หมดอายุ service refresh สิทธิ์หรือ clear session. ไม่ส่งชื่อหน้าต่าง-โดยไม่มี session. Backend รับ machine-only heartbeat แล้วอัปเดต lastSeen; AgentLog สร้างเมื่อมี session ที่ใช้ได้



รูปที่ 10.2: Heartbeat และกิจกรรม (แผนภาพลำดับ)

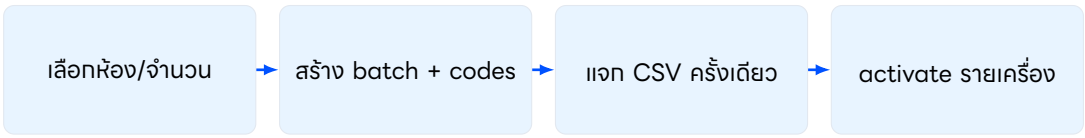
หมายเหตุ Agent login ปัจจุบันไม่ล็อก macOS/Windows/Linux desktop ก่อนใช้งาน เครื่องยังใช้ได้โดยไม่ login Agent. นี่เป็นข้อจำกัดของการควบคุมการใช้เครื่อง และช่วงนั้นไม่มี activity ที่ผูกผู้ใช้

หลักฐานใน repository: agent/src/machine_service.py;agent/src/session_ui.py;agent/src/config.py;web/src/app/api/agent/heartbeat/route.ts;web/src/app/api/agent/auth/route.ts

บทที่ 11

Batch Enrollment และรหัสเปิดใช้งาน

SUPER_ADMIN สร้าง Deployment Batch ที่ /admin/system. Backend ตรวจห้อง active และจำนวน 1–100, จอง machine code รูปแบบ <roomCode>-PC<sequence>, สร้าง code หกหลักด้วย randomInt, hash ก่อนเก็บ และกำหนดหมดอายุ 7 วัน. Transaction ลองแก้ collision ตามจำนวนครั้งที่กำหนด. CSV ที่ browser ได้ครั้งเดียวมีคอลัมน์ batch_id, room_code, machine_code, activation_code, expires_at ตามลำดับ



รูปที่ 11.1: Batch Enrollment (แผนภาพอธิบาย)

1. เลือกห้องและจำนวนเครื่องใน System Control; ตรวจเลขที่จองและวันหมดอายุ
2. เก็บ CSV ในที่จำกัดสิทธิ์; แจกหนึ่งรหัสต่อหนึ่งเครื่อง ไม่ส่งผ่านช่องทางสาธารณะ
3. ติดตั้งแพ็กเกจ OS ที่ถูกต้องแล้วกรอก code ใน UI; ตรวจ machineCode/roomCode
4. ดู deployment progress ใน System Control และสถานะ Online ใน Admin Tracking

รหัสถูกใช้ได้ครั้งเดียว. การ revoke batch/code มีผลกับ code ที่ยังไม่ถูกใช้; เครื่องที่ activate แล้วต้อง deactivate/uninstall แยก. Failed attempts ถูกบันทึกและมี durable activation throttle. หาก reinstall หลังถอนซึ่งลบ config ต้องออก code ใหม่; ไม่ใช้ code เก่าซ้ำ

หลักฐานใน repository: web/src/lib/agent-deployment-batches.ts;web/src/lib/agent-activation.ts;web/src/lib/agent-activation-throttle.ts;web/src/app/api/agent/activate/route.ts;web/src/app/admin/system/system-client.tsx

บทที่ 12

ติดตั้งและถอน Agent บน Windows

เป้าหมาย Windows 10/11 x64. Installer คือ ElegantAgentSetup_0.5.0_x64.exe จาก Admin download หรือ artifact ที่ตรวจ SHA-256 แล้ว. Build บน Windows ใช้ Python 3.12, PyInstaller และ Inno Setup 6; ไม่ใช่ MSI. Installer ลง service ElegantSolutionsMachine และ UI auto-start ใน HKLM Run เมื่อผู้ใช้ Windows login

1. รับ EXE จากผู้ดูแล; เทียบ hash กับ release ที่แจกอย่างปลอดภัย; login ด้วยบัญชี Administrator
2. Run as administrator; รอ service แล้วเปิด UI. กรอก activation code ของเครื่องนี้
3. ตรวจ service/Run registration ด้วย PowerShell ด้านล่าง; ตรวจเครื่อง Online ใน Admin Tracking ภายในประมาณ 10–30 วินาที
4. ทดสอบ Agent login ด้วยบัญชีที่มี entitlement ปัจจุบัน, foreground activity, logout, machine-only heartbeat และ reboot

```
Get-Service ElegantSolutionsMachine
sc.exe qc ElegantSolutionsMachine
Get-ItemProperty 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Run' -Name ElegantAgentUI
Test-Path "$env:ProgramData\ElegantSolution\Agent\config.json"
```

Program files อยู่ C:\Program Files\ElegantSolution \Agent ; config/API key/logs อยู่ C:\ProgramData \ElegantSolution \Agent . ถ้า UI ไม่ขึ้น ให้ตรวจ registry Run และ Windows Event Viewer. **ยังไม่มีหลักฐานติดตั้ง Windows จริง** จากชุดข้อมูลนี้

ถอนและอัปเดต

ใน Admin Tracking สร้าง maintenance code ชนิด UNINSTALL_AGENT ของเครื่องนั้นก่อน แล้วเปิด C:\Program Files\ElegantSolution \Agent \uninstall .bat แบบ Administrator. Script ตรวจ code กับ backend, หยุด/ลบ service และ registry, ลบ process/งานเก่าที่ระบุ, ลบ hosts entries เฉพาะส่วนที่ Agent จัดการ แล้วลบ ProgramData/Program Files ของ Agent. Backend deactivate machine เมื่ออนุญาต code; หากขั้นลบในเครื่องล้มเหลวหลังยืนยัน code ให้หยุดและตรวจสถานะ ไม่ใช้คำสั่งลบสุ่ม. อัปเดตโดย installer ใหม่ทับ; config ใน ProgramData ไม่อยู่ใน payload

หลักฐานใน repository: agent/build.bat;agent/setup.iss;agent/setup.bat;agent/uninstall.bat;agent/WINDOWS_INSTALLATION.md

บทที่ 13

ติดตั้งและถอน Agent บน macOS

เลือก PKG ให้ตรง uname -m: arm64 สำหรับ Apple Silicon หรือ x86_64 สำหรับ Intel. PKG รุ่นพัฒนา 0.5.0 ยัง unsigned/unnotarized; ให้ผู้ดูแลประเมินแหล่งที่มาและ hash ก่อนอนุญาตเฉพาะแพ็กเกจที่เชื่อถือ อย่าปิด Gatekeeper ทั้งระบบ. การแจกจ่ายถาวรควรลงลายเซ็นและ notarize

```
uname -m
shasum -a 256 -c elegant-agent_0.5.0_darwin_$(uname -m).pkg.sha256
sudo installer -pkg elegant-agent_0.5.0_darwin_$(uname -m).pkg -target /
sudo launchctl print system/com.elegantsolution.agent.machine
launchctl print gui/$(id -u)/com.elegantsolution.agent.session
```

PKG วาง binary ที่ /Library/Application Support/ElegantSolution/bin/; LaunchDaemon ทำงานตอน boot, LaunchAgent เริ่ม UI เมื่อ login desktop; config/API key ใน /Library/Application Support/ElegantSolution/Agent/. หลัง activate ตรวจ Admin Tracking → Agent login ที่มี entitlement → activity → logout → reboot. การอ่านชื่อ app/window ผ่าน System Events ต้องได้สิทธิ์ Automation/Accessibility; ถ้าไม่ได้สิทธิ์ activity อาจเป็น null แต่ heartbeat ต่อได้

หลักฐานเครื่องจริงและถอน

ผู้ใช้รายงานว่า macOS PKG activate สำเร็จ, หลัง reboot UI เปิดเองไม่ต้อง activate ซ้ำ, Admin Tracking เห็นเครื่องเดิม Online โดยไม่ login นักศึกษาและไม่มีรายการซ้ำ. ต่อมารัน uninstaller แล้วได้ Elegant Agent removed; pkgutil --pkg-info ไม่พบ receipt และ launchctl print ไม่พบสอง service. นี่คือหลักฐานจากผู้ใช้ ไม่ใช่การตรวจเครื่องโดยผู้เขียนคู่มือโดยตรง. ผล Agent login/activity ยังไม่ยืนยัน

```
# สร้าง UNINSTALL_AGENT maintenance code ใน Admin Tracking ก่อน
sudo '/Library/Application Support/ElegantSolution/bin/uninstall-agent.sh'
pkgutil --pkg-info com.elegantsolution.agent
sudo launchctl print system/com.elegantsolution.agent.machine
launchctl print gui/$(id -u)/com.elegantsolution.agent.session
```

Uninstaller ตรวจ code แล้วหยุด LaunchDaemon/LaunchAgent ลบ plist, binary, Agent และ Bridge, ลบ PKG receipt. ไม่ลบไฟล์ /var/log/elegant-agent-service.log และ /var/log/elegant-agent-service.err ตาม script. Backend deactivate machine เมื่อรับ code ถอนสำเร็จ; หาก local script ล้มเหลวภายหลัง อาจเกิดสถานะ deactivate ก่อนลบครบ. Reinstall หลังถอนต้องใช้ activation code ใหม่; update PKG ทับเก็บ config

หลักฐานใน repository: agent/build-macos.sh;agent/macos/pkg-scripts/postinstall;agent/macos/uninstall-agent.sh;agent/MACOS_INSTALLATION.md;web/src/app/api/agent/maintenance/authorize/route.ts

บทที่ 14

ติดตั้งและถอน Agent บน Ubuntu

เป้าหมาย Ubuntu 22.04/24.04 amd64. DEB เป็น frozen binary; machine service ใช้ elegant-agent.service ผ่าน systemd และ UI ใช้ XDG autostart หลัง graphical login. X11 อาจอ่าน foreground ผ่าน EWMH; Wayland ส่ง app/title เป็น null เมื่อ API ไม่รองรับ จึงต้องทดสอบ desktop จริง

```
uname -m
sha256sum -c elegant-agent_0.5.0_amd64.deb.sha256
sudo apt install ./elegant-agent_0.5.0_amd64.deb
systemctl status elegant-agent.service --no-pager
systemctl is-enabled elegant-agent.service
journalctl -u elegant-agent.service -n 50 --no-pager
```

เข้า desktop แล้วกรอก activation code ใน UI. ตรวจ Online, Agent login/activity, logout/heartbeat และ reboot. Config/API key อยู่ /var/lib/elegant-agent/config.json; token เชื่อม UI อยู่ /run/elegant-agent/token; binary ที่ /usr/lib/elegant-agent/. **ยังไม่มีหลักฐานติดตั้ง Ubuntu จริง** ในข้อมูลที่ตรวจ

ถอน อัปเดต และ legacy

สร้าง UNINSTALL_AGENT maintenance code ใน Admin Tracking แล้วรัน sudo elegant-agent-uninstall; script ตรวจ code แล้ว apt-get purge -y elegant-agent. ตรวจ systemctl status elegant-agent.service, dpkg -s elegant-agent และไฟล์ config ตามนโยบาย package postrm. อัปเดตด้วย sudo apt install ./<new-version>.deb. การย้าย-จาก user-level agent เดิมมี sudo elegant-agent-migrate-legacy <desktop-username> ซึ่งคัดลอก config เฉพาะ-เมื่อปลายทางว่างและปิด autostart เก่า; สำรอง config อย่างปลอดภัยก่อน

หลักฐานใน repository: agent/build-linux.sh;agent/linux/elegant-agent.service;agent/linux/elegant-agent.desktop;agent/linux/uninstall-agent.sh;agent/linux/postrm;agent/LINUX_INSTALLATION.md

บทที่ 15

บำรุงรักษาและแก้ปัญหา

ใช้รูปแบบ อาการ → สาเหตุที่เป็นไปได้ → ตรวจ → แก้ → ยืนยัน. เริ่มจากการตรวจแบบอ่านอย่างเดียวก่อนเสมอ

อาการ	สาเหตุ	ตรวจ	แก้และยืนยัน
UI ไม่เปิด	graphical login/autostart ไม่ทำงาน	launchctl/Run/XDG entry	เปิด UI ตามคู่มือ OS แล้วทดสอบ login ใหม่
Machine service หยุด	service startup/permission	launchctl, Get-Service, systemctl	ตรวจ log, restart ด้วย maintenance code, ดู heartbeat
Activate ไม่ได้	code ผิด/หมดอายุ/ใช้แล้ว, URL/TLS	code status, HTTPS, response 403/429	ออก code ใหม่ให้เครื่องถูกต้อง; รอตาม Retry-After
เครื่อง Offline	heartbeat หยุด, backend/เน็ตล่ม	service status, DNS/TLS, /api/liveness	กู้การเชื่อมต่อ; รอ heartbeat รอบใหม่
Agent login ถูกปฏิเสธ	บัญชีไม่ approved/ไม่มี entitlement	ตรวจ role, room, CLASS/BOOKING/token และเวลา	แก้ข้อมูลต้นเหตุแล้ว login ใหม่
ไม่มีชื่อแอป	ไม่ login Agent, macOS permission, Wayland	ตรวจ session/UI/permission	ให้สิทธิ์ OS ตามนโยบาย; ตรวจ activity ใหม่
Maintenance code ถูกปฏิเสธ	ผิด action/เครื่อง, หมด 5 นาที, ใช้แล้ว	ตรวจรายการใน Admin Tracking	สร้าง code ใหม่โดยไม่ deactivate เครื่องก่อน
Checksum ไม่ตรง	artifact ผิด/ดาวน์โหลดเสีย	เทียบ SHA-256 จาก release ที่เชื่อถือ	หยุดติดตั้ง ดาวน์โหลดใหม่

ข้อควรระวัง หาก uninstall ได้รับอนุญาตจาก backend แล้วแต่ลบไฟล์ในเครื่องไม่สำเร็จ ให้บันทึก error และสถานะ service ก่อนดำเนินการต่อ เพราะ backend อาจ deactivate machine ไปแล้ว. อย่าเผยแพร่ config, API key, CSV หรือ log ที่มีข้อมูลบุคคล

หลักฐานใน repository: agent/src/maintenance_cli.py;agent/src/api_client.py;web/src/app/api/agent/activate/route.ts;web/src/app/api/agent/maintenance/authorize/route.ts

ภาค III

อ้างอิงเทคนิคสำหรับผู้พัฒนา

บทที่ 16

สถาปัตยกรรมและเทคโนโลยี

เว็บใช้ Next.js 16.1.6, React 19.2.3, NextAuth 5 beta, Prisma 7.3 และ PostgreSQL 16 ตาม package/schema/runbook. Face AI เป็น FastAPI + InsightFace รุ่นโมเดลที่กำหนดใน service. OCR ใช้ Tesseract.js 5.1.1 ที่ browser ผ่าน CDN และ parser ใน TypeScript. Tracking Agent เป็น Python core แยก service/UI; PyInstaller package สำหรับ Windows, macOS และ Linux. CI สร้างสี่ artifacts แต่ CI build ไม่-ใช่หลักฐาน end-to-end บนเครื่องจริง เส้นทาง request: Browser login/web actions ผ่าน NextAuth และ server



รูปที่ 16.1: องค์ประกอบซอฟต์แวร์ (แผนภาพอธิบาย)

action; Gate ส่ง QR/ภาพไป API; Next.js ตรวจข้อมูล/สิทธิ์ฐานข้อมูล; Face API เรียก service ภายใน. Agent ใช้ machine key แยกจาก web session. PM2/Nginx/Cloudflare อยู่ในชั้น deploy ไม่ใช่โค้ด business logic

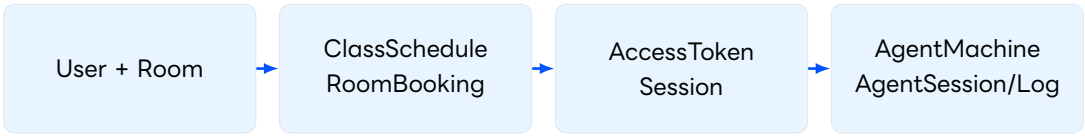
หลักฐานใน repository: web/package.json;web/prisma/schema.prisma;web/ai-service/main.py;agent/README.md;.github/workflows/agent-packages.yml

บทที่ 17

ฐานข้อมูลและความสัมพันธ์

Prisma ใช้ User เป็นบัญชี/role/status; Room เป็นห้องและนโยบาย; ClassSchedule ผูก user/room/semester; RoomBooking ผูกผู้ขอ/ห้อง/วัน/status โดย RoomBookingSlot ระบุช่วง และ RoomBookingParticipant บันทึก snapshot ผู้มีสิทธิ์. AccessToken คือ legacy token ช่วงเวลา; Session บันทึก check-in/out; FaceProfile เก็บ embedding ของ user

ด้าน Agent มี AgentMachine ผูกห้องและ machine code/API key; AgentSession ผูก machine/user/room พร้อม token hash และ expiry; AgentLog บันทึกกิจกรรม; AgentActivationCode, AgentDeploymentBatch, AgentMachineSequence, AgentActivationThrottle, AgentMaintenanceCode และ AgentAuditEvent จัดการวงจร-เปิดใช้และตรวจสอบย้อนหลัง Constraint ที่ควรรู้: Room.roomCode และ AgentMachine.machineCode เป็น unique;



รูปที่ 17.1: ER แบบย่อ: ห้องและการใช้งาน (ภาพอธิบาย)

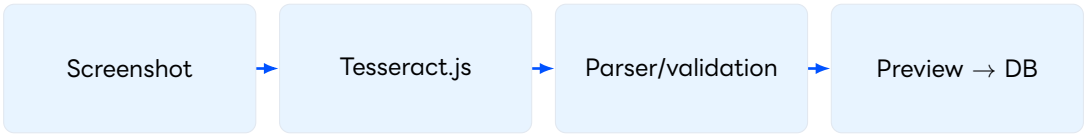
booking participant unique ต่อคู่ booking/user; slot unique ต่อ booking/slot; availability window unique ต่อ room/day/slot. การลบห้อง/ผู้ใช้มีข้อจำกัดจาก FK; ห้ามลบข้อมูลจริงเพื่อแก้ migration อย่างรวดเร็ว. ชุด migration ที่เคยซ้อมมี 17 รายการ และ production upgrade เคยรายงาน 17/17; คู่มือนี้ไม่ได้อ่านฐานข้อมูล production สด

หลักฐานใน repository: web/prisma/schema.prisma;docs/PRODUCTION_PM2_DEPLOYMENT.md;web/src/lib/room-entitlement.ts

บทที่ 18

กระบวนการ OCR ตารางเรียน

Browser โหลด Tesseract.js จาก CDN, สร้าง worker, อ่านภาพแต่ละไฟล์เป็นข้อความ จากนั้น parseOcrBatch แยกบรรทัด/กลุ่มวิชา ตรวจชื่อ, semester, day, course code, section, class type, เวลา, room และเทียบ academic slot. Parser รวมแถวซ้ำและให้สถานะ ready/warning/error. UI แสดง preview และ raw OCR/debug เพื่อให้ผู้ใช้ตรวจก่อน import ผ่าน server action; server ตรวจข้อมูลอีกครั้งก่อนบันทึก ตัวอย่างสมมติ: ภาพมีข้อความ CS101 LAB Mon



รูปที่ 18.1: OCR pipeline (ภาพอธิบาย)

09:00-12:00 LAB-A; preview ควรแสดงรหัสวิชา, ประเภท LAB, วัน/เวลา และห้องที่อ่านได้ ผู้ใช้ต้องแก้ถ้าภาพเบลอหรือ parser จับ slot ไม่ได้. ต้องทดสอบด้วยภาพที่ได้รับอนุญาต; OCR ไม่ตรวจความจริงของตารางจากมหาวิทยาลัยโดยตรง

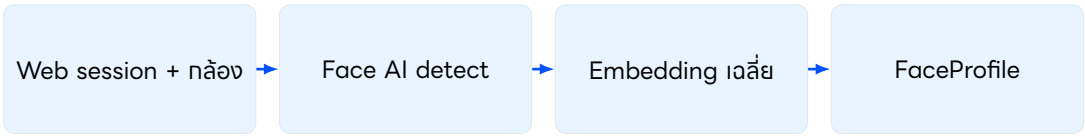
หลักฐานใน repository: web/src/components/ocr-timetable-panel.tsx;web/src/lib/ocr-timetable.ts;web/src/actions/ocr-timetable.ts;web/src/lib/academic-slots.ts

บทที่ 19

Face AI และการยืนยันใบหน้า

Enrollment

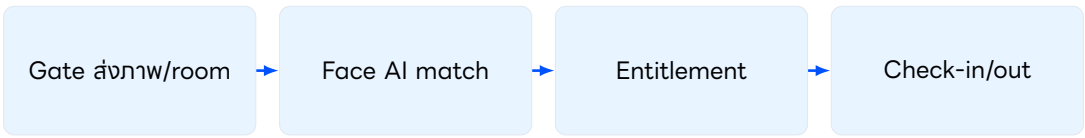
POST /api/face/enroll ต้องมี web session และปฏิเสธ GUEST. UI เก็บภาพจากกล้อง 1–3 ภาพ; API ส่ง user_id และ-ภาพ base64 ไป Face AI /enroll. Service ใช้ InsightFace ตรวจหน้า เลือกใบหน้าที่ detection score สูงสุดในแต่ละภาพ ตรวจขนาด/คุณภาพ และเฉลี่ย embedding ที่ได้. Next.js upsert FaceProfile และบันทึก log. โค้ดที่ตรวจไม่ได้ยืนยันว่า-ภาพมีหน้าเดียวเสมอ; การเลือกใบหน้าที่คะแนนสูงสุดอาจไม่ตรงคนที่ตั้งใจ



รูปที่ 19.1: Face enrollment (ภาพอธิบาย)

Verification

POST /api/face/verify รับภาพกับ room_code; server ตรวจห้อง active, ส่ง embeddings ของ non-guest users ให้ Face AI เปรียบเทียบ 1:N ด้วย threshold ค่าเริ่ม 0.73 หรือ FACE_THRESHOLD. เมื่อ match แล้วโหลด user, ตรวจ staffOnly, และสำหรับ STUDENT/GUEST ตรวจ entitlement ห้องปัจจุบัน; ADMIN/SUPER_ADMIN ไม่ผ่าน entitlement check นี้ตามโค้ด. หากอนุญาตจะเรียก checkInAction; ถ้ามี active session เดิมจึง checkOutAction. Log เก็บผลการยืนยัน ภาพไม่ถูกต้อง, ไม่พบหน้า, ไม่มี profile หรือ service ล่มต้องปฏิเสธ/คืน error ตาม API. ไม่พบ



รูปที่ 19.2: Face verification (ภาพอธิบาย)

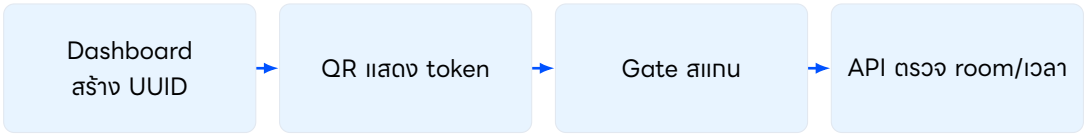
liveness หรือ anti-spoofing ใน flow ที่ตรวจ; ไม่อ้างว่ากันภาพถ่ายได้. การตรวจผ่านซอฟต์แวร์ไม่ยืนยันกลอนไฟฟ้าเปิด-จริง. Embedding เป็นข้อมูลชีวมิติ ต้องจำกัดสิทธิ์และกำหนดนโยบายการเก็บ/ลบ

หลักฐานใน repository: web/src/app/api/face/enroll/route.ts;web/src/app/api/face/verify/route.ts;web/ai-service/main.py;web/src/actions/sessions.ts

บทที่ 20

QR, access token และ Gate

Dashboard สร้าง token ด้วย crypto.randomUUID() ผ่าน generateAccessTokenAction แล้วใช้ library qrcode สร้างภาพ QR จาก ค่า token โดยตรง ไม่ใช่ URL. Gate page /gate/[roomCode] ใช้ html5-qrcode อ่าน QR แล้วเรียก server action verifyGateAccessAction โดยตรง; route POST /api/gate/verify เป็นทางเข้า API แยกที่เรียก action เดียวกัน. Validation ตรวจ token ตรง room และอยู่ในช่วง validFrom/validTo; ถ้าไม่ใช่ token ที่ใช้ได้จะลองตีความ input เป็น studentId เพื่อหาตารางเรียนของห้อง/เวลานั้น



รูปที่ 20.1: QR/token flow (ภาพอธิบาย)

ความต่างที่ต้องไม่สับสน

- การสร้าง QR เป็นการแสดง token; ไม่เท่ากับการสร้าง booking และไม่ใช่ code ใช้ครั้งเดียวโดยอัตโนมัติ
- verifyAccessTokenAction ตรวจช่วงเวลาและ room; token สามารถใช้ซ้ำในช่วงเวลาที่กำหนดตามโค้ดที่ตรวจ ไม่พบการ consume หลังสแกน
- verifyGateAccessAction มีเส้นทาง student ID fallback; นี่ไม่ใช่ QR token และใช้เงื่อนไขตารางเรียนเฉพาะทาง
- resolveRoomEntitlement ใช้ legacy token เฉพาะที่ออกให้ user/room นั้น; group token ไม่ขยายสิทธิ์สู่สมาชิกใน resolver แม้ Gate validation ของ token กลุ่มมี branch แยก
- การตอบ valid: true จาก server action/API Gate ไม่เท่ากับการบันทึก Session เสมอ; หน้า Gate จึงเรียก checkInAction/checkOutAction ต่อและตรวจผล. โค้ดไม่มีหลักฐานการควบคุมฮาร์ดแวร์ประตูแบบครบวงจร

หลักฐานใน repository: web/src/actions/access-tokens.ts;web/src/actions/groups.ts;web/src/app/(app)/dashboard/dashboard-client.tsx;web/src/app/gate/[roomCode]/page.tsx;web/src/app/api/gate/verify/route.ts

บทที่ 21

รหัสและ token สำคัญ

ชนิด	สร้างโดย/รูปแบบ	อายุ	เก็บ/ตรวจ	เพิกถอน/ข้อสังเกต
Room code	Admin กำหนด; unique ใน Room	จนเปลี่ยน	DB, room lookup	ปิด room ได้
Machine code	sequence ต่อห้อง, -PCnn	ตาม machine	unique ใน DB	deactivate machine
Batch activation	randomInt, 6 หลัก	7 วัน	hash, consumedAt	revoke unused; ใช้ครั้งเดียว
Code เดี่ยว	randomInt, 6 หลัก	10 นาที	hash, consumedAt	cancel unused
Machine API key	DB default/generated	จน deactivate	config เครื่อง + DB	deactivate/ถอน; ไม่ใส่ installer
Agent session token	randomBytes(32) base64url	ถึง entitlement end	SHA-256 hash ใน DB	logout/expiry/revoke
Maintenance code	randomInt, 6 หลัก	5 นาที	hash, ผูก action/machine	ใช้ครั้งเดียว
Access token QR	crypto. randomUUID()	validFrom–validTo	token ใน DB/QR	ไม่พบ consume เมื่อสแกน
Booking ID/batch ID	Prisma default cuid()	ตลอด record	primary key	สถานะ/ลบตามกฎ DB

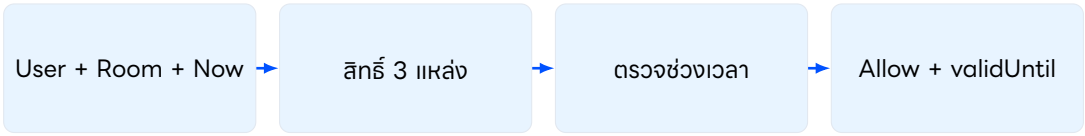
Hash activation/maintenance ใช้ helper hashAgentOneTimeCode ที่ผูก secret ของ server; session token เก็บ hash. API key ของ machine อยู่ใน config ที่ root/service อ่านได้และส่งใน header x-agent-key. Rate limit activation ใช้ durable throttle; Agent login/maintenance มีการจำกัดอัตราแยก. อย่านำ code ตัวอย่างในคู่มือไปใช้จริง

หลักฐานใน repository: web/src/lib/agent-activation.ts;web/src/lib/agent-deployment-batches.ts;web/src/lib/agent-session.ts; web/src/lib/agent-maintenance.ts;web/src/actions/access-tokens.ts;web/prisma/schema.prisma

บทที่ 22

Room Availability และ Entitlement

Availability สำหรับการจองตรวจ room active, bookingEnabled, availability windows, closures, class schedule, booking เดิม, slot, capability, capacity, วันและเวลา. AUTO อนุมัติอัตโนมัติเมื่อผ่านกฎ; MANUAL ให้สถานะ pending รอ Admin. การจองแบบกลุ่มใช้ participant snapshot และ advisory locks กันการจองทับ/แข่งกัน; การเปลี่ยนสมาชิก Group ภายหลังไม่เปลี่ยนสิทธิ์ booking เดิม CLASS ใช้ semester ที่ active, วันและช่วงเรียน โดยเริ่มก่อนชั้นเรียน 30



รูปที่ 22.1: Entitlement decision (ภาพอธิบาย)

นาที. BOOKING ต้อง approved และ user อยู่ใน RoomBookingParticipant, เวลาอยู่ภายในช่วง slot. LEGACY_TOKEN ต้องออกให้ user และ room นี้และยังไม่หมดอายุ. หากหลายแหล่งทับกันให้สิทธิ์ถึงเวลาสิ้นสุดที่ไกลที่สุด. Agent login ใช้ resolver นี้; Face verify ใช้กับ STUDENT/GUEST. ตัวอย่างสมมติ: เรียน 09:00–12:00 ที่ห้อง LAB-A จะเริ่มสิทธิ์ CLASS 08:30 ถึงก่อน 12:00 ตามเวลา Asia/Bangkok

หลักฐานใน repository: web/src/lib/availability.ts;web/src/lib/room-booking.ts;web/src/lib/room-booking-rules.ts;web/src/lib/room-entitlement.ts;web/src/app/api/agent/auth/route.ts

บทที่ 23

Agent API และ Monitoring

Path	Method	ยืนยันตัวตน	หน้าที่/ข้อมูลสำคัญ
/api/agent/activate	POST	code 6 หลัก + throttle	hostname/platform → machine code, room, API key
/api/agent/auth	POST	machine key + email/password	entitlement → sessionToken/expiresAt
/api/agent/heartbeat	POST	machine key; session optional	lastSeen/version; log activity เมื่อ session valid
/api/agent/config	GET	machine key	blacklist/room/platform
/api/agent/session/refresh	POST	machine key + session	ต่ออายุหาก entitlement ยังมี
/api/agent/session/logout	POST	machine key + session	revoke user session
/api/agent/maintenance/authorize	POST	machine key + code	ตรวจ action; ถอนแล้ว deactivate
/api/tracking/snapshot	GET	Admin web session	สถานะเครื่อง/ผู้ใช้/กิจกรรมล่าสุด
/api/tracking/machine-logs	GET	Admin web session	log ล่าสุด 50 แถวตาม machineId
/api/admin/agent/installers/[platform]	GET	Admin web session	EXE/PKG/DEB ที่ stage แล้ว

Heartbeat ทุก 10 วินาทีอัปเดต AgentMachine.lastSeen; dashboard ถือ Online เมื่อ active และ lastSeen ใหม่กว่า 30 วินาที. Activity ล่าสุดแสดงเฉพาะเมื่อเครื่อง Online, มี Agent session ปัจจุบัน และ log เป็นผู้ใช้เดียวกันหลังเวลาออก session. Missing/invalid session token ส่ง 401; machine-only heartbeat ยังเป็น 200 และไม่สร้าง AgentLog. API ตัวอย่างไม่ควรฝัง key จริง

ข้อมูลและการตอบกลับที่ต้องรู้

Activation รับ activationCode, hostname, platform; สำเร็จส่ง machineCode, roomCode, apiKey. รูปแบบผิดได้ 400, code ผิด/หมดอายุ/ใช้แล้ว/ถูกยกเลิกได้ 403 และ throttle ได้ 429 พร้อม Retry-After. Auth รับ email/password คู่ machine key; บัญชีไม่ approved หรือรหัสผิดได้ 401, ห้องไม่ active หรือไม่มี entitlement ได้ 403. Session token ต้องส่งใน x-agent-session เมื่อรายงาน activity. Maintenance รับ code/action; code ผิดได้ 403, machine key ผิดได้ 401 และ throttle ได้ 429. Snapshot และ machine logs ต้องมี Admin web session; installer route อาจตอบ 503 เมื่อยังไม่ตั้ง directory หรือ 404 เมื่อยังไม่ได้ stage file. ตัวอย่าง request ด้านล่างเป็นข้อมูลสมมติทั้งหมด

```
POST /api/agent/heartbeat
X-Agent-Key: <machine-key-redacted>
Content-Type: application/json
```

```
{"agentVersion":"0.5.0"}
```

หลักฐานใน repository: web/src/app/api/agent/*/route.ts;web/src/app/api/tracking/snapshot/route.ts;web/src/lib/agent-health.ts;agent/src/api_client.py

บทที่ 24

ความปลอดภัย ความเป็นส่วนตัว และข้อจำกัด

Backend แยก web session/role, machine key และ Agent session. Activation/maintenance code ถูก hash, ใช้ครั้งเดียว มีวันหมดอายุและ throttle. Machine config อยู่ในพื้นที่สิทธิ์จำกัด; UI ติดต่อ machine service ด้วย local token. HTTPS ใช้ที่ public endpoint. Admin ดู machine/user/activity ตามสิทธิ์; ข้อมูลซื่อหน้าต่างอาจมีข้อมูลส่วนตัว ควรกำหนดการแจ้งผู้ใช้ ระยะเก็บ และสิทธิ์เข้าถึงก่อนขยายใช้งานจริง

สถานะ	ข้อเท็จจริง/งานที่ต้องตรวจ
IMPLEMENTED	Agent machine heartbeat แยกจาก student login, QR gate path, Face AI match, room entitlement, batch codes, admin snapshot
AUTOMATED TESTED	รายงานก่อนหน้าระบุ Agent 19, Web 549, DB 166 และ build/CI ผ่าน; ไม่รันชุดนี้ซ้ำเพื่อทำเอกสาร
VERIFIED ON REAL HARDWARE	ผู้ใช้รายงาน macOS install/activate/Online/reboot UI และ machine identity เดิม รวมถึงถอนด้วย maintenance code และตรวจ receipt/launchctl
NOT YET VERIFIED	Student Agent login/activity/logout บนเครื่องจริง; Windows/Ubuntu install/reboot จริง; ประตูฮาร์ดแวร์; macOS title capture หลังให้สิทธิ์
PARTIAL IMPLEMENTATION	เว็บไซต์ blacklist ยังไม่พิสูจน์การบังคับใช้ครบสาม OS; Group QR UI ปิด; Contact/Survey ไม่ควรนำเสนอเป็น workflow production
NOT IMPLEMENTED	Agent 0.5.0 ไม่บังคับ OS login ก่อนใช้ desktop; ไม่พบ liveness/anti-spoofing ใน Face flow

คู่มือ Agent เก่าบางไฟล์ระบุ backend /api/agent/activate เป็น 404 ณ 11 พ.ย. 2026 ซึ่งเป็นหลักฐานประวัติ ไม่ใช่สถานะหลัง deployment ที่รายงานในวันถัดมา. การติดตั้ง/activate macOS ที่ผู้ใช้รายงานพิสูจน์ว่า endpoint ใช้งานได้ในช่วงทดลองนั้น แต่ไม่ได้ยืนยันทุก endpoint หรือทุก OS. การถอน macOS script ไม่ลบ log ใน /var/log; การออก maintenance code ถอนจะ deactivate machine ก่อนลบ local เสร็จ จึงต้องมีขั้นตรวจเมื่อเกิด partial failure

หลักฐานใน repository: agent/EXAM_CHEAT_SHEET.md;agent/macos/uninstall-agent.sh;web/src/app/api/agent/maintenance/authorize/route.ts;web/src/lib/agent-health.ts

ภาคผนวก A

เช็กลิสต์ปฏิบัติงาน

ทุกข้อเป็นเกณฑ์ที่ผู้ดูแลควรลงชื่อ/บันทึกผลในระบบงานของตนเอง ไม่ใช่หลักฐานว่าผ่านแล้ว คอลัมน์ผลให้ทำเครื่องหมาย-เมื่อมีหลักฐานจริง

งาน	เกณฑ์ตรวจเสร็จ
Backend ใหม่	ตรวจ Node/pnpm/PM2/PostgreSQL, SHA release, .env โดยไม่เปิดเผยค่า, backup+restore test, migration status, build, smoke 3001, public health
Super Admin ครั้งแรก	บัญชี SUPER_ADMIN ที่ได้รับอนุญาต login ได้; หน้า /admin/system แสดงเฉพาะ role นี้; บันทึกเจ้าของบัญชี
สร้างห้อง	roomCode ไม่ซ้ำ, active, capacity, capabilities, bookingEnabled, approvalPolicy, guestAccess/staffOnly, windows/closures ถูกต้อง
นำเข้าตาราง	semester/day/slot/room/user ตรงข้อมูลจริง; preview/รายการผิดพลาดตรวจแล้ว
Deployment Batch	ห้อง active; จำนวน 1–100; CSV ดาวน์โหลดครั้งเดียว; แจก code ตาม machine code; เก็บ CSV จำกัดสิทธิ์
ติดตั้งหนึ่งเครื่อง	hash installer ตรง; OS/architecture ตรง; service/UI เริ่ม; activate; room/machine code ตรง; heartbeat Online
ติดตั้ง 30 เครื่อง	แบ่ง batch ตามห้อง; บันทึก asset/machine code; ตรวจไม่ใช้ code ซ้ำ; ตรวจ Online ครบและไม่มีเครื่องซ้ำ
Agent login	บัญชี approved มี entitlement ณ เวลานั้น; session ผูกเครื่อง/ห้อง/ผู้ใช้; logout แล้ว heartbeat ยังอยู่
ทดสอบ OCR	ภาพสมมติที่อ่านได้; preview, warning, duplicate และ DB import ถูกต้อง
ทดสอบ Face	ลงทะเบียนด้วยผู้มีสิทธิ์; ตรวจ match/no match และ entitlement; ไม่อ้าง liveness/ประตู
ทดสอบ QR	UUID token ใน QR; gate สแกน; room/time validation; ตรวจผล check-in/out แยก
ถอน Windows	maintenance code ตรงเครื่อง/action; service/Run/ไฟล์หาย; backend inactive
ถอน macOS	receipt/LaunchDaemon/LaunchAgent หาย; backend inactive; ตรวจ log ที่ script ไม่ลบตามนโยบาย
ถอน Ubuntu	package/service/config ตาม postrm; backend inactive; legacy autostart ไม่เหลือ
อัปเดต Agent	hash/version ใหม่; config เดิมอยู่; service/UI เริ่ม; machine code เดิม; heartbeat กลับมา
สำรอง DB	pg_dump -Fc, SHA-256, pg_restore --list, restore แยกตามรอบทดสอบ

งาน	เกณฑ์ตรวจเสร็จ
กู้ service	ระบุเหตุ/เวลา/ผู้รับผิดชอบ; rollback app ตาม runbook; ตรวจ PM2, Nginx, DB, HTTPS, Agent heartbeat

ภาคผนวก B

คำสั่งอ้างอิงแบบอ่านอย่างเดียว

เลือกชุดคำสั่งตาม OS ของเครื่องที่จะตรวจ ไม่รัน installer/uninstaller บนเครื่องที่ไม่ใช่เป้าหมาย ไม่คัดลอกผลที่มี API key หรือข้อมูลส่วนบุคคลลง ticket สาธารณะ

```
# macOS หลังติดตั้ง
sudo launchctl print system/com.elegantsolution.agent.machine
launchctl print gui/$(id -u)/com.elegantsolution.agent.session
pkgutil --pkg-info com.elegantsolution.agent

# Ubuntu หลังติดตั้ง
systemctl status elegant-agent.service --no-pager
systemctl is-enabled elegant-agent.service
journalctl -u elegant-agent.service -n 50 --no-pager

# Backend health แบบไม่ต้องมี credential
curl --fail --silent --show-error \
  https://elegant.fafromh0me.com/api/liveness
```

```
# Windows PowerShell หลังติดตั้ง
Get-Service ElegantSolutionsMachine
sc.exe qc ElegantSolutionsMachine
Get-ItemProperty 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Run' -Name ElegantAgentUI
```

ถ้า launchctl print หลังถอนรายงาน Could not find service และ pkgutil รายงาน No receipt นั่นสอดคล้องกับการถอนสำเร็จ; ตรวจ Admin Tracking แยกอีกครั้ง

ภาคผนวก C

อภิธานศัพท์และสถานะ

คำ	ความหมายในระบบนี้
Web session	ตัวตนผู้ใช้เว็บจาก NextAuth JWT; แยกจาก Agent session
Machine identity	machine code/API key หลัง activation; ใช้ machine heartbeat
Agent session	token ที่ผูกผู้ใช้-เครื่อง-ห้องและหมดอายุตาม entitlement
Entitlement	สิทธิ์ใช้ห้อง ณ เวลาใช้งานจาก CLASS, BOOKING หรือ LEGACY_TOKEN
Heartbeat	ข้อความจาก service เพื่ออัปเดต lastSeen; ไม่ยืนยันผู้ใช้
Online	เครื่อง active และ lastSeen ภายใน 30 วินาที
Deployment Batch	ชุด code ที่จองให้ห้องและจำนวนเครื่อง; code ใช้ครั้งเดียว
Maintenance code	code หกหลักผูกเครื่อง/action มีอายุ 5 นาที
Gate	หน้าเว็บ/endpoint ตรวจ QR หรือใบหน้า; ไม่ใช่หลักฐานฮาร์ดแวร์เปิด
Preview	รายการ OCR ก่อนบันทึกที่ผู้ใช้ต้องตรวจ

ภาคผนวก D

บัญชีประเด็นเอกสารและแหล่งอ้างอิง

ข้อแตกต่างและสิ่งที่ควรตรวจต่อ

1. agent/*INSTALLATION.md และ agent/EXAM_CHEAT_SHEET.md ยังกล่าวถึง production endpoint 404 เมื่อ 11 พ.ย. 2026; สถานะนั้นเก่ากว่าผลการติดตั้ง/activate macOS ที่ผู้ใช้รายงาน
2. Agent 0.5.0 ไม่บังคับ OS login ก่อน desktop ใช้งาน; หากต้องการควบคุมเครื่องแล็บจริงต้องออกแบบ OS-level policy และทดสอบบนเครื่องที่จัดการได้
3. web/prisma/seed.ts สร้างบัญชีตัวอย่าง ADMIN พร้อม credential ที่ฝังใน source แต่ไม่มี SUPER_ADMIN; ต้องวางขั้น provision ครั้งแรกที่ปลอดภัยก่อนใช้ระบบจริง และห้ามรัน seed นี้บน production
4. QR access token legacy และ room entitlement ใหม่มีเงื่อนไขไม่เหมือนกัน; ทบทวน policy ก่อนขยายการใช้งาน Gate จริง
5. Room Finder ซ่อนห้อง staffOnly จาก Guest แต่ generateAccessTokenAction ตรวจ guestAccess โดยไม่ตรวจ staffOnly โดยตรง; ทบทวนการบังคับสิทธิ์ Gate/token ก่อนใช้งานจริง
6. Face AI เลือกหน้าที่ detection score สูงสุดเมื่อภาพมีหลายหน้า; ยังไม่มี liveness/anti-spoofing ใน flow ที่ตรวจ
7. macOS uninstaller ไม่ลบ log สองไฟล์ใน /var/log; backend deactivate machine ในขั้น authorize ก่อน local cleanup จบ
8. Windows/Ubuntu ยังต้องติดตั้งและทดสอบ reboot/activity จริง; macOS foreground activity และ student login ก็ยังไม่มีผลทดสอบที่ส่งมา

แผนที่หลักฐานใน repository

- โครงสร้างและ role: web/prisma/schema.prisma, web/src/lib/permissions.ts, web/src/lib/auth.ts
- ห้องและการจอง: web/src/lib/availability.ts, web/src/lib/room-booking.ts, web/src/lib/room-entitlement.ts
- OCR/Face/Gate: web/src/lib/ocr-timetable.ts, web/src/app/api/face/*, web/src/actions/access-tokens.ts
- Agent: agent/src/machine_service.py, agent/src/session_ui.py, agent/build-macos.sh, agent/build-linux.sh, agent/build.bat
- Deployment: docs/PRODUCTION_PM2_DEPLOYMENT.md; Agent API: web/src/app/api/agent/*

หมายเหตุ ภาพทั้งหมดในคู่มือเป็นแผนภาพต้นฉบับจากโค้ด ไม่ใช่ภาพหน้าจอ production; ตัวอย่างรหัส/ข้อมูลใช้ข้อความสมมติ ไม่มี backup, secret หรือข้อมูลใบหน้าจริงอยู่ในเอกสาร